

Serialization : Code-Along (CodeGrade)

 <https://github.com/learn-co-curriculum/python-p4-v2-serialization>  <https://github.com/learn-co-curriculum/python-p4-v2-serialization/issues/new>

Learning Goals

- Use SQLAlchemy-Serializer to convert SQLAlchemy objects into dictionaries.

Key Vocab

- **Serialization:** a process to convert programmatic data such as a Python object to a sequence of bytes that can be shared with other programs, computers, or networks.
- **Deserialization:** the reverse process, converting a sequence of bytes back to programmatic data.
- **SQLAlchemy-Serializer:** A powerful tool for serializing data in Python using the SQLAlchemy ORM.

Introduction

Serialization allows us to convert data such as an Python object to a sequence of bytes that can be shared with other programs, computers, or networks. Deserialization is the reverse process, converting a sequence of bytes back to one or more Python objects.

SQLAlchemy-Serializer is a powerful tool for serializing data in Python using the SQLAlchemy ORM. It provides an easy and efficient way to convert SQLAlchemy database models into dictionaries or other data formats.

We've seen how to manually create a dictionary to map object attribute names as keys to values. In this lesson, we'll use the SQLAlchemy-Serializer library to generate this dictionary for a given model class.

Setup

This lesson is a code-along, so fork and clone the repo.

Run `pipenv install` to install the dependencies and `pipenv shell` to enter your virtual environment before running your code.

```
$ pipenv install
$ pipenv shell
```

Change into the `server` directory and configure the `FLASK_APP` and `FLASK_RUN_PORT` environment variables:

```
$ cd server
$ export FLASK_APP=app.py
$ export FLASK_RUN_PORT=5555
```

The commands `flask db init` and `flask db migrate` have already been run. Run the following command to initialize the database from the existing migration script:

```
$ flask db upgrade head
```

Run the following command to seed the table with sample data:

```
$ python seed.py
```

Use the Flask shell to confirm 10 random random pets have been added to the database (your results may differ if you need to rerun the migration sequence):

```
$ flask shell
>>> Pet.query.all()
```

```
[<Pet 1, Jodi, Chicken>, <Pet 2, Cheyenne, Dog>, <Pet 3, Christina, Turtle>, <Pet 4, Jessica, Turtle>, <Pet 5, Sharc  
>>> exit()
```

SerializerMixin

Within the `server` folder, open `models.py` and you'll notice a new import at the top:

```
# models.py  
from sqlalchemy_serializer import SerializerMixin
```

When a model class inherits from the `SerializerMixin`, it gains a range of methods for serializing and deserializing data. These methods include `to_dict()`, which converts the model object into a dictionary.

In `models.py`, we'll need to reconfigure our model to inherit from `SerializerMixin`. Don't worry though- this only requires a small amount of new code, and we won't have to run new migrations afterward. Update the `Pet` class header to add `SerializerMixin` as a superclass:

```
# models.py  
# imports, config  
  
class Pet(db.Model, SerializerMixin):  
    ...
```

Make sure you are in the `server/` directory and run `flask shell` to start manipulating our models. Let's retrieve a `Pet` record and then run its brand new method, `to_dict()`:

```
$ flask shell  
>>> from models import Pet  
>>> Pet.query.all()
```

```
[<Pet 1, Jodi, Chicken>, <Pet 2, Cheyenne, Dog>, <Pet 3, Christina, Turtle>, <Pet 4, Jessica, Turtle>, <Pet 5, Sharc
>>> pet1 = Pet.query.first()
>>> pet1
<Pet 1, Jodi, Chicken>
>>> pet1.to_dict()
```

Wow, the `to_dict()` method that `Pet` inherits from `SerializerMixin` returns a dictionary mapping each attribute name to the current instance's values!

`to_dict()`

`to_dict()` is a simple method: it takes a SQLAlchemy object, turns its columns into dictionary keys, and turns its column values into dictionary values.

Updating our Flask Application that returns a JSON reponse

The previous version of our server had to manually create the pet dictionary from the query result:

```
@app.route('/pets/<int:id>')
def pet_by_id(id):
    pet = Pet.query.filter(Pet.id == id).first()

    if pet:
        body = {'id': pet.id,
                'name': pet.name,
                'species': pet.species}
        status = 200
    else:
        body = {'message': f'Pet {id} not found.'}
        status = 404
```

```
return make_response(body, status)
```

Now that our `Pet` model inherits from `SerializerMixin`, we can just call `to_dict()` on a `Pet` instance to generate the dictionary. Edit `app.py` to add the following route and view:

```
@app.route('/pets/<int:id>')
def pet_by_id(id):
    pet = Pet.query.filter(Pet.id == id).first()

    if pet:
        body = pet.to_dict()
        status = 200
    else:
        body = {'message': f'Pet {id} not found.'}
        status = 404

    return make_response(body, status)
```

Save and test the app with a valid id using the URL <http://127.0.0.1:5555/pets/5> ↗ (<http://127.0.0.1:5555/pets/5>), along with a non-existent id such the URL <http://127.0.0.1:5555/pets/1000> ↗ (<http://127.0.0.1:5555/pets/1000>).

In a similar fashion, we can use `to_dict()` to create the dictionary for each pet returned when we query by species. Add the following route and view to `app.py`:

```
@app.route('/species/<string:species>')
def pet_by_species(species):
    pets = [] # array to store a dictionary for each pet
    for pet in Pet.query.filter_by(species=species).all():
        pets.append(pet.to_dict())
    body = {'count': len(pets),
            'pets': pets}
```

```
}  
return make_response(body, 200)
```

Save and test the app with an species that matches some pets using the URL <http://127.0.0.1:5555/species/Dog> , along with a species that matches no pets such the URL <http://127.0.0.1:5555/species/Whale> , along with a species that matches no pets such the URL <http://127.0.0.1:5555/species/Whale>.

Conclusion

SQLAlchemy-Serializer is a helpful tool that helps programmers turn complex data into simpler, portable formats. SQLAlchemy-Serializer makes it easy to convert a Python object into a dictionary, which is then easily converted to JSON.

Solution Code

```
from flask_sqlalchemy import SQLAlchemy  
from sqlalchemy import MetaData  
from sqlalchemy_serializer import SerializerMixin  
  
metadata = MetaData()  
  
db = SQLAlchemy(metadata=metadata)  
  
class Pet(db.Model, SerializerMixin):  
    __tablename__ = 'pets'  
  
    id = db.Column(db.Integer, primary_key=True)  
    name = db.Column(db.String)  
    species = db.Column(db.String)  
  
    def __repr__(self):
```

```
return f'<Pet {self.id}, {self.name}, {self.species}>'
```

```
# server/app.py
```

```
#!/usr/bin/env python3
```

```
from flask import Flask, make_response
```

```
from flask_migrate import Migrate
```

```
from models import db, Pet
```

```
app = Flask(__name__)
```

```
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///app.db'
```

```
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
```

```
app.json.compact = False
```

```
migrate = Migrate(app, db)
```

```
db.init_app(app)
```

```
@app.route('/')
```

```
def index():
```

```
    body = {'message': 'Welcome to the pet directory!'}
```

```
    return make_response(body, 200)
```

```
@app.route('/pets/<int:id>')
```

```
def pet_by_id(id):
```

```
    pet = Pet.query.filter(Pet.id == id).first()
```

```
    if pet:
```

```
        body = pet.to_dict()
```

```
        status = 200
```

```
    else:
```

```

        body = {'message': f'Pet {id} not found.'}
        status = 404

    return make_response(body, status)

@app.route('/species/<string:species>')
def pet_by_species(species):
    pets = [] # array to store a dictionary for each pet
    for pet in Pet.query.filter_by(species=species).all():
        pets.append(pet.to_dict())
    body = {'count': len(pets),
            'pets': pets
           }
    return make_response(body, 200)

if __name__ == '__main__':
    app.run(port=5555, debug=True)

```

```

#!/usr/bin/env python3
#server/seed.py
from random import choice as rc
from faker import Faker

from app import app
from models import db, Pet

with app.app_context():

    # Create and initialize a faker generator
    fake = Faker()

    # Delete all rows in the "pets" table

```



```
Pet.query.delete()

# Create an empty list
pets = []

species = ['Dog', 'Cat', 'Chicken', 'Hamster', 'Turtle']

# Add some Pet instances to the list
for n in range(10):
    pet = Pet(name=fake.first_name(), species=rc(species))
    pets.append(pet)

# Insert each Pet in the list into the "pets" table
db.session.add_all(pets)

# Commit the transaction
db.session.commit()
```

Resources

- [Quickstart - Flask-SQLAlchemy](https://flask-sqlalchemy.palletsprojects.com/en/2.x/quickstart/#) ➞ <https://flask-sqlalchemy.palletsprojects.com/en/2.x/quickstart/#>
- [Flask-Migrate](https://flask-migrate.readthedocs.io/en/latest/) ➞ <https://flask-migrate.readthedocs.io/en/latest/>
- [SQLAlchemy-Serializer](https://pypi.org/project/SQLAlchemy-serializer/) ➞ <https://pypi.org/project/SQLAlchemy-serializer/>

This tool needs to be loaded in a new browser window

Load Serialization : Code-Along (CodeGrade) in a new window